

Lending Club Credit Default — Exploratory Data Analysis & PCA

45-952 End-to-End Business Analytics — Credit Default Case

This notebook performs exploratory data analysis (EDA) and Principal Component Analysis (PCA) on the Lending Club loans dataset (`loanskpi.csv`), following the same workflow style used in the course's `Untitled0` (Carseats EDA) and `Proteins` (PCA) notebooks.

Data source: `data/loanskpi.csv` in this repository, described in `docs/LoansDataDescr.pdf` .

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from IPython.display import display

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
```

1. Load the data

```
In [2]: data_path = "../data/loanskpi.csv"
df = pd.read_csv(data_path)
print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns")
display(df.head())
```

Shape: 9578 rows x 17 columns

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fi
0	1	debt_consolidation	0.1189	829.10	11.350407	19.48	7
1	1	credit_card	0.1071	228.22	11.082143	14.29	7
2	1	debt_consolidation	0.1357	366.86	10.373491	11.63	6
3	1	debt_consolidation	0.1008	162.34	11.350407	8.10	7
4	1	credit_card	0.1426	102.92	11.299732	14.97	6

2. Dataset info, descriptive statistics, and missing values

```
In [3]: df.info()
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 9578 entries, 0 to 9577  
Data columns (total 17 columns):  
#   Column                Non-Null Count  Dtype    
---  ---  
0   credit.policy          9578 non-null   int64    
1   purpose                9578 non-null   str       
2   int.rate               9578 non-null   float64   
3   installment            9578 non-null   float64   
4   log.annual.inc         9578 non-null   float64   
5   dti                    9578 non-null   float64   
6   fico                   9578 non-null   int64     
7   days.with.cr.line      9578 non-null   float64   
8   revol.bal              9578 non-null   int64     
9   revol.util             9578 non-null   float64   
10  inq.last.6mths         9578 non-null   int64     
11  delinq.2yrs            9578 non-null   int64     
12  pub.rec                9578 non-null   int64     
13  default                9578 non-null   int64     
14  tot.payment            9578 non-null   float64   
15  principal              9578 non-null   float64   
16  interest               9578 non-null   float64   
dtypes: float64(9), int64(7), str(1)  
memory usage: 1.2 MB
```

```
In [4]: display(df.describe(include='all').T)
```

	count	unique	top	freq	mean	
credit.policy	9578.0	NaN	NaN	NaN	0.80497	0.39
purpose	9578	7	debt_consolidation	3957	NaN	
int.rate	9578.0	NaN	NaN	NaN	0.12264	0.02
installment	9578.0	NaN	NaN	NaN	319.089413	207.07
log.annual.inc	9578.0	NaN	NaN	NaN	10.932117	0.61
dti	9578.0	NaN	NaN	NaN	12.606679	6.8
fico	9578.0	NaN	NaN	NaN	710.846314	37.97
days.with.cr.line	9578.0	NaN	NaN	NaN	4560.767197	2496.93
revol.bal	9578.0	NaN	NaN	NaN	16913.963876	33756.18
revol.util	9578.0	NaN	NaN	NaN	46.799236	29.01
inq.last.6mths	9578.0	NaN	NaN	NaN	1.577469	2.20
delinq.2yrs	9578.0	NaN	NaN	NaN	0.163708	0.54
pub.rec	9578.0	NaN	NaN	NaN	0.062122	0.26
default	9578.0	NaN	NaN	NaN	0.160054	0.36
tot.payment	9578.0	NaN	NaN	NaN	11487.218877	7454.56
principal	9578.0	NaN	NaN	NaN	7861.686703	4927.31
interest	9578.0	NaN	NaN	NaN	3625.532173	2666.26

```
In [5]: missing = pd.DataFrame(df.isnull().sum(), columns=['Missing Values'])
missing['% Missing'] = (missing['Missing Values'] / len(df) * 100).round(2)
display(missing)
```

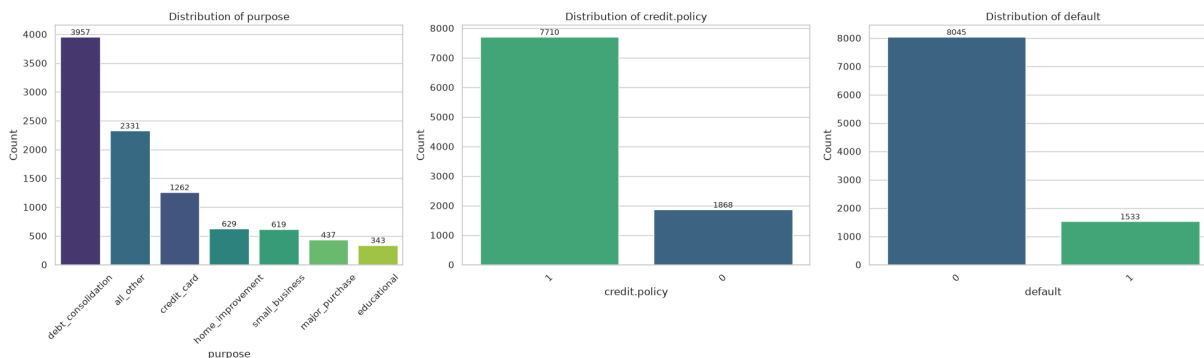
	Missing Values	% Missing
credit.policy	0	0.0
purpose	0	0.0
int.rate	0	0.0
installment	0	0.0
log.annual.inc	0	0.0
dti	0	0.0
fico	0	0.0
days.with.cr.line	0	0.0
revol.bal	0	0.0
revol.util	0	0.0
inq.last.6mths	0	0.0
delinq.2yrs	0	0.0
pub.rec	0	0.0
default	0	0.0
tot.payment	0	0.0
principal	0	0.0
interest	0	0.0

3. Categorical variable distributions

`purpose` is the only text-categorical field; `credit.policy` and `default` are binary flags that behave like categories.

```
In [6]: categorical_cols = ['purpose', 'credit.policy', 'default']

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
for ax, col in zip(axes, categorical_cols):
    order = df[col].value_counts().index
    sns.countplot(data=df, x=col, order=order, hue=col, palette='viridis', 1
    ax.set_title(f'Distribution of {col}')
    ax.set_xlabel(col)
    ax.set_ylabel('Count')
    ax.tick_params(axis='x', rotation=45)
    for p in ax.patches:
        ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width()/2,
        ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('../output/categorical_distributions.png', dpi=150, bbox_inches=
plt.show()
```



```
In [7]: for col in categorical_cols:
        print(f"--- {col} ---")
        display((df[col].value_counts(normalize=True) * 100).round(2).rename('%'))
        print()
```

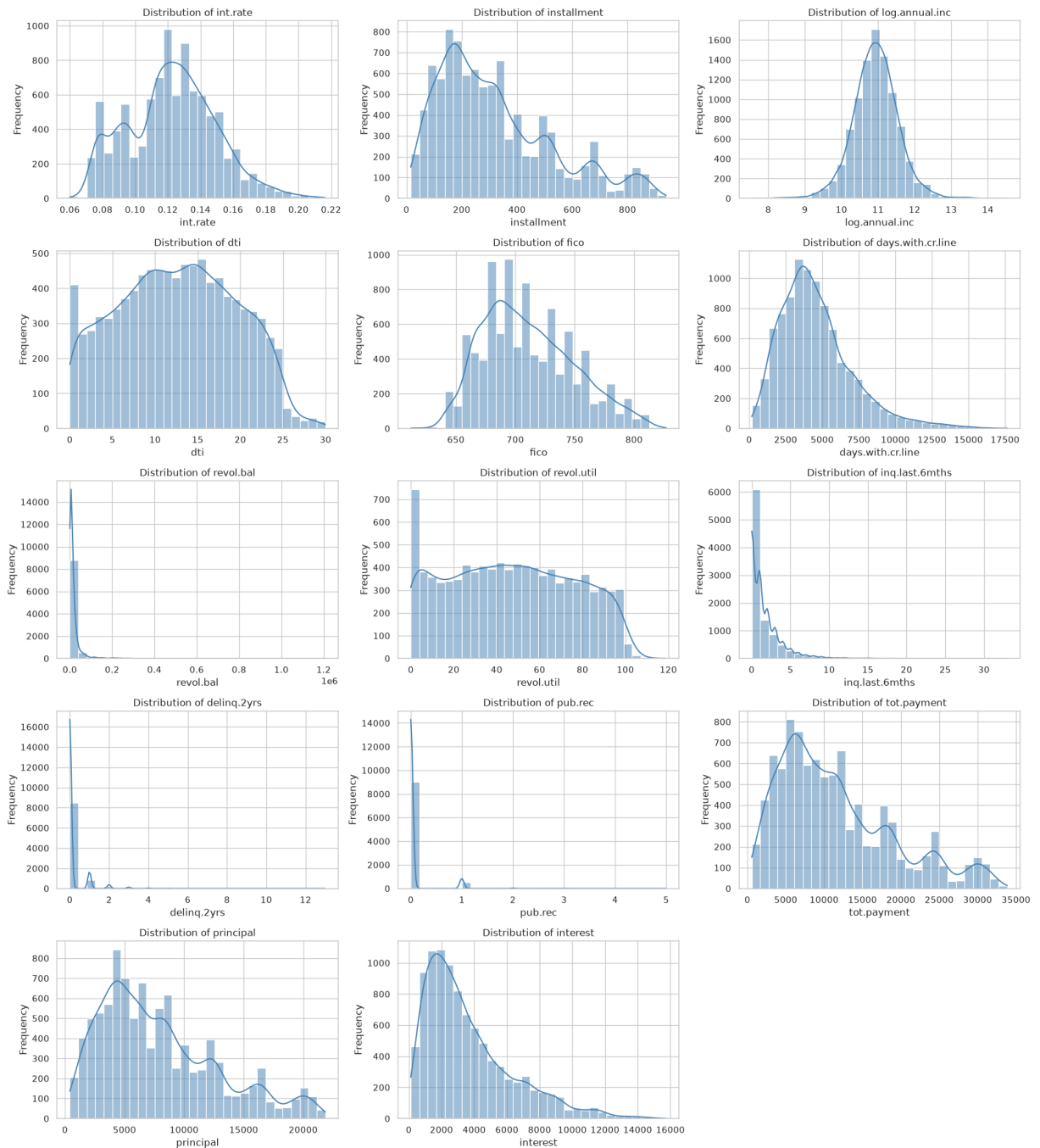
```
--- purpose ---
purpose
debt_consolidation    41.31
all_other             24.34
credit_card           13.18
home_improvement       6.57
small_business         6.46
major_purchase         4.56
educational           3.58
Name: % of loans, dtype: float64
--- credit.policy ---
credit.policy
1      80.5
0      19.5
Name: % of loans, dtype: float64
--- default ---
default
0      83.99
1      16.01
Name: % of loans, dtype: float64
```

4. Numerical variable distributions

```
In [8]: numeric_cols = ['int.rate', 'installment', 'log.annual.inc', 'dti', 'fico',
                        'days.with.cr.line', 'revol.bal', 'revol.util', 'inq.last.6m',
                        'delinq.2yrs', 'pub.rec', 'tot.payment', 'principal', 'inter

n_cols = 3
n_rows = int(np.ceil(len(numeric_cols) / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes = axes.flatten()
for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, bins=30, color='steelblue', ax=axes[i])
    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')
for j in range(len(numeric_cols), len(axes)):
    fig.delaxes(axes[j])
```

```
plt.tight_layout()
plt.savefig('../output/numerical_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



5. Correlation matrices and heatmaps

Two views are shown:

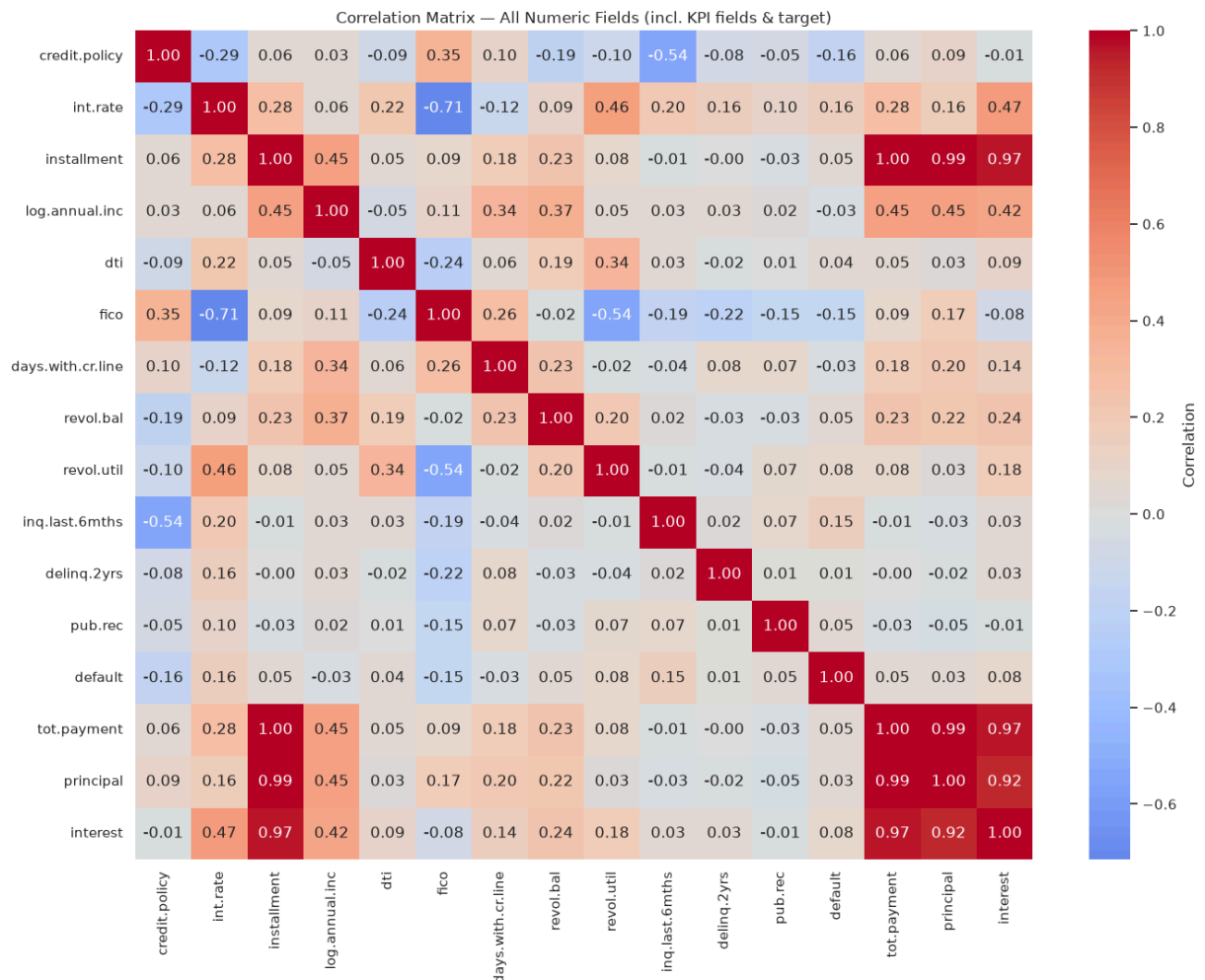
1. **Full correlation matrix** — all numeric fields, including the computed KPI fields (`tot.payment` , `principal` , `interest`) and the default target. Note that `int.rate` is expected to correlate with risk indicators such as `fico` and `revol.util` (per the data description), and the KPI fields are

deterministic functions of `installment / int.rate`, so they will show very strong correlations by construction.

2. **Predictor-only correlation matrix** — restricted to the borrower/loan fields known *at the time of the funding decision* (excludes the post-outcome KPI fields, which is also the feature set used for PCA below).

```
In [9]: full_numeric = df.select_dtypes(include=[np.number])
corr_full = full_numeric.corr()

plt.figure(figsize=(14, 11))
sns.heatmap(corr_full, annot=True, cmap='coolwarm', fmt=".2f", center=0,
            cbar_kws={'label': 'Correlation'})
plt.title('Correlation Matrix — All Numeric Fields (incl. KPI fields & target)')
plt.tight_layout()
plt.savefig('../output/correlation_matrix_full.png', dpi=150, bbox_inches='tight')
plt.show()
```

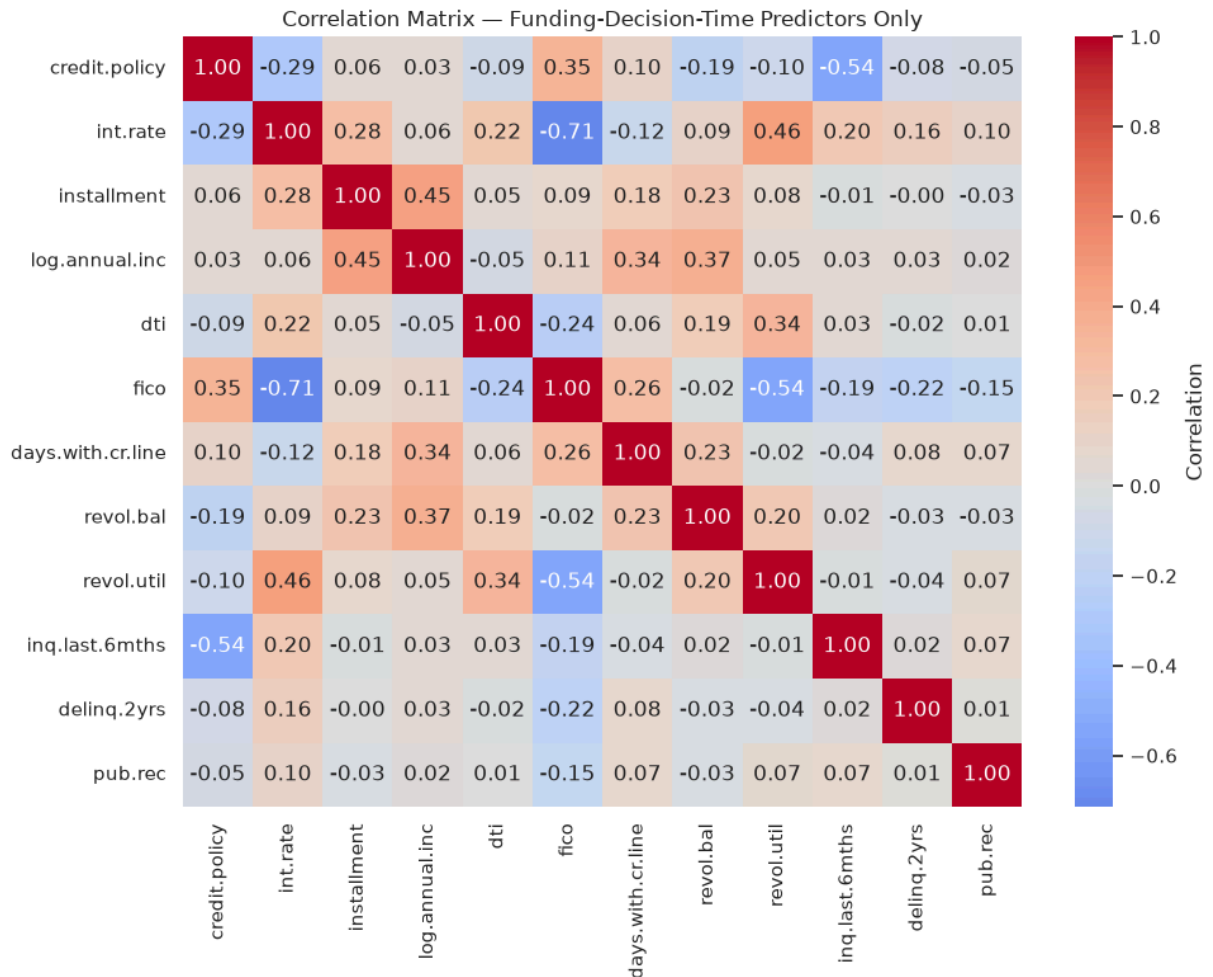


```
In [10]: predictor_cols = ['credit.policy', 'int.rate', 'installment', 'log.annual.ir',
                           'fico', 'days.with.cr.line', 'revol.bal', 'revol.util',
                           'inq.last.6mths', 'delinq.2yrs', 'pub.rec']

corr_predictors = df[predictor_cols].corr()

plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr_predictors, annot=True, cmap='coolwarm', fmt=".2f", center=
            cbar_kws={'label': 'Correlation'})
plt.title('Correlation Matrix – Funding-Decision-Time Predictors Only')
plt.tight_layout()
plt.savefig('../output/correlation_matrix_predictors.png', dpi=150, bbox_inches=
plt.show()
```



6. Principal Component Analysis (PCA)

PCA is run on the funding-decision-time predictors (numeric fields available when the loan is funded, plus one-hot-encoded `purpose`). The computed KPI fields (`tot.payment`, `principal`, `interest`) and the `default` target are excluded from the PCA feature set to avoid leakage, since those fields are either deterministic transforms of `installment` / `int.rate` or the outcome itself.

```
In [11]: pca_features = predictor_cols.copy()
X = pd.get_dummies(df[pca_features + ['purpose']].drop(columns=['credit.poli
            columns=['purpose'], drop_first=True)

# Reorder for clarity: keep credit.policy as-is (already 0/1)
X = X.astype(float)

scaler = StandardScaler()
```



```

X_scaled = scaler.fit_transform(X)

pca = PCA()
pca.fit(X_scaled)

explained_variance_ratio = pca.explained_variance_ratio_
cumulative_explained_variance = explained_variance_ratio.cumsum()

print('Explained variance ratio per component:')
for i, ratio in enumerate(explained_variance_ratio):
    print(f'PC{i+1}: {ratio*100:.2f}%')

print('\nCummulative explained variance:')
for i, cum in enumerate(cumulative_explained_variance):
    print(f'PC{i+1}: {cum*100:.2f}%')

X_pca = pca.transform(X_scaled)
pca_df = pd.DataFrame(X_pca, columns=[f'PC{i+1}' for i in range(X_pca.shape[1])])
pca_df['default'] = df['default'].values
display(pca_df.head())

```

Explained variance ratio per component:

PC1: 15.32%
PC2: 11.52%
PC3: 8.94%
PC4: 7.29%
PC5: 6.52%
PC6: 6.20%
PC7: 5.80%
PC8: 5.70%
PC9: 5.54%
PC10: 5.34%
PC11: 4.80%
PC12: 4.06%
PC13: 3.47%
PC14: 2.69%
PC15: 2.18%
PC16: 2.01%
PC17: 1.66%
PC18: 0.95%

Cumulative explained variance:

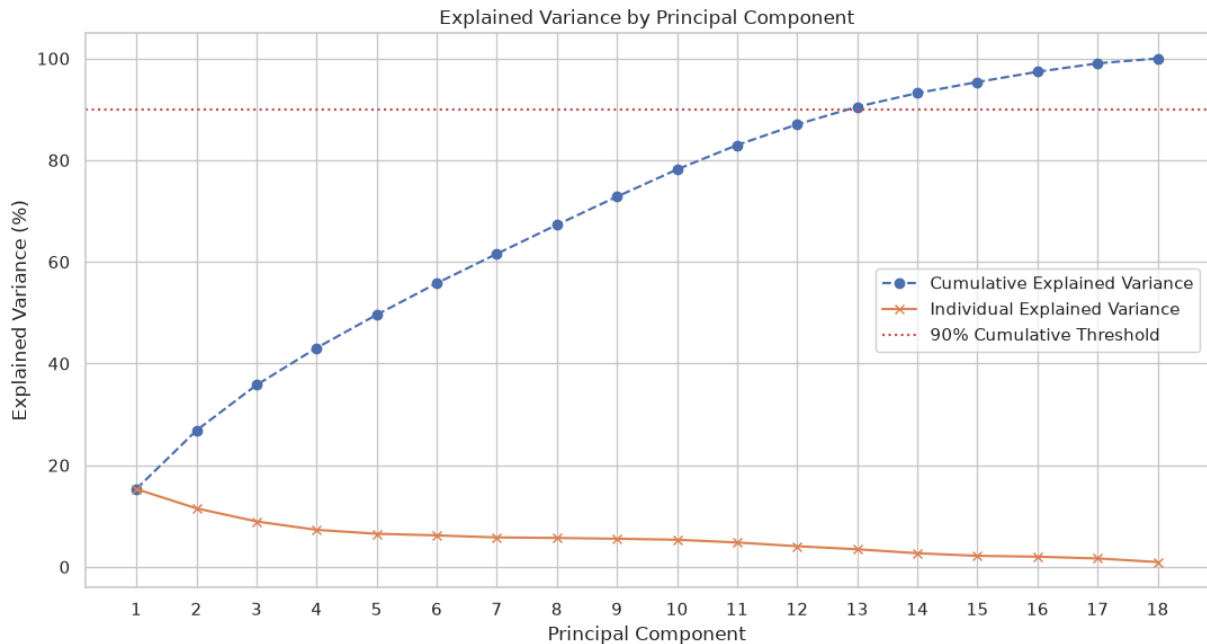
PC1: 15.32%
PC2: 26.83%
PC3: 35.78%
PC4: 43.07%
PC5: 49.59%
PC6: 55.79%
PC7: 61.58%
PC8: 67.28%
PC9: 72.83%
PC10: 78.17%
PC11: 82.97%
PC12: 87.04%
PC13: 90.51%
PC14: 93.19%
PC15: 95.37%
PC16: 97.38%
PC17: 99.05%
PC18: 100.00%

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	
0	-0.153119	2.161846	-1.682934	-0.792218	-0.299740	-0.476571	0.106498	0.1
1	0.080075	0.167831	-0.473069	2.907152	0.568846	-0.175311	-0.119369	-0.6
2	-0.225922	-0.653155	-1.103501	-0.935059	0.010484	-0.032410	-0.060020	-0.0
3	0.124700	-0.027540	-1.164359	-0.334469	-0.479078	-0.308380	0.031350	-0.2
4	-0.546866	-0.351705	0.116370	2.349499	1.454897	1.484990	-0.053194	0.3

```
In [12]: num_components = range(1, len(cumulative_explained_variance) + 1)

plt.figure(figsize=(11, 6))
plt.plot(num_components, cumulative_explained_variance * 100, marker='o', li
        label='Cumulative Explained Variance')
```

```
plt.plot(num_components, explained_variance_ratio * 100, marker='x', linestyle='dashed', label='Individual Explained Variance')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance (%)')
plt.title('Explained Variance by Principal Component')
plt.grid(True)
plt.xticks(num_components)
plt.axhline(y=90, color='r', linestyle=':', label='90% Cumulative Threshold')
plt.legend(loc='best')
plt.tight_layout()
plt.savefig('../output/pca_explained_variance.png', dpi=150, bbox_inches='tight')
plt.show()
```



Biplot of the first two Principal Components

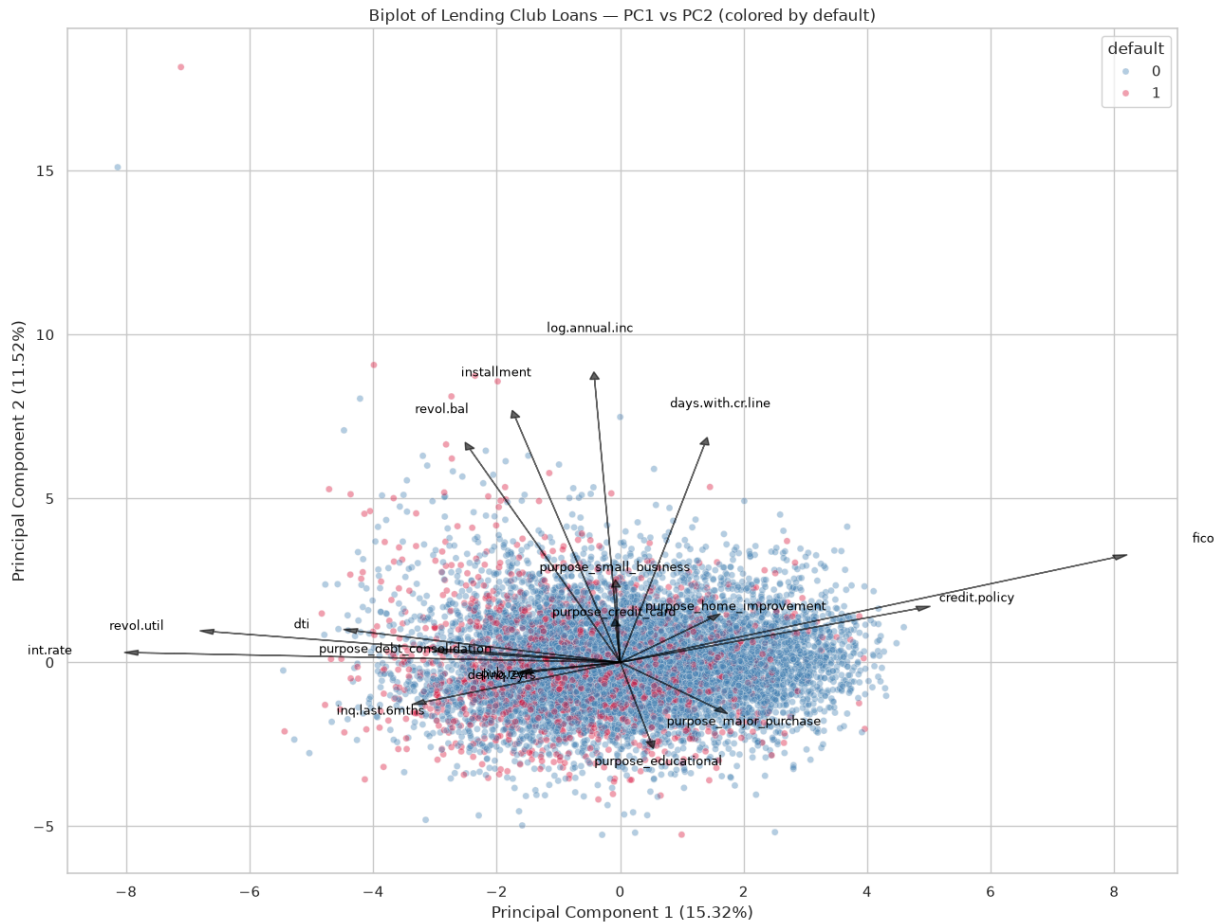
```
In [13]: loadings = pca.components_
feature_names = X.columns

fig = plt.figure(figsize=(13, 10))
sns.scatterplot(x=pca_df['PC1'], y=pca_df['PC2'], hue=pca_df['default'],
                palette={0: 'steelblue', 1: 'crimson'}, alpha=0.4, s=25)

scale = np.abs(pca_df[['PC1', 'PC2']]).max().max() * 0.9
for i, feature in enumerate(feature_names):
    plt.arrow(0, 0, loadings[0, i] * scale, loadings[1, i] * scale,
              color='black', alpha=0.6, head_width=0.15, length_includes_head=True)
    plt.text(loadings[0, i] * scale * 1.15, loadings[1, i] * scale * 1.15, feature,
              color='black', ha='center', va='center', fontsize=9)

plt.xlabel(f'Principal Component 1 ({explained_variance_ratio[0]*100:.2f}%)')
plt.ylabel(f'Principal Component 2 ({explained_variance_ratio[1]*100:.2f}%)')
plt.title('Biplot of Lending Club Loans - PC1 vs PC2 (colored by default)')
plt.legend(title='default', loc='best')
plt.grid(True)
plt.tight_layout()
```

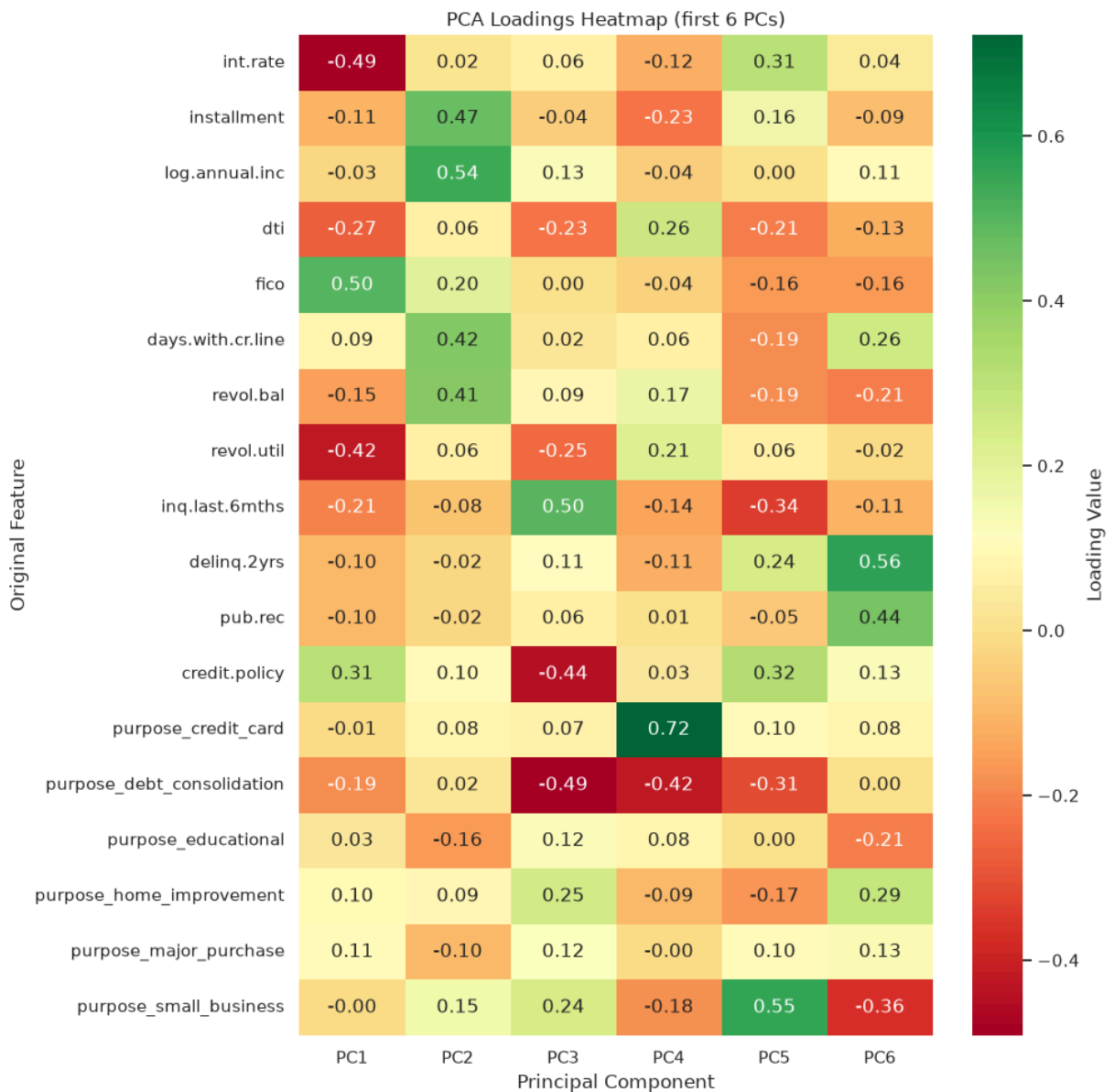
```
plt.savefig('../output/pca_biplot.png', dpi=150, bbox_inches='tight')
plt.show()
```



PCA Loadings Heatmap

```
In [14]: loadings_df = pd.DataFrame(pca.components_, columns=feature_names,
                                     index=[f'PC{i+1}' for i in range(pca.n_component

n_show = min(6, loadings_df.shape[0])
plt.figure(figsize=(10, 10))
sns.heatmap(loadings_df.iloc[:n_show].T, annot=True, cmap='RdYlGn', fmt=".2f",
            cbar_kws={'label': 'Loading Value'})
plt.title(f'PCA Loadings Heatmap (first {n_show} PCs)')
plt.xlabel('Principal Component')
plt.ylabel('Original Feature')
plt.tight_layout()
plt.savefig('../output/pca_loadings_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



7. Default rate by first two principal components

A quick check of how the target variable (`default`) varies across the PCA space, and the top absolute loadings for PC1 and PC2 for interpretation.

```
In [15]: for pc in ['PC1', 'PC2', 'PC3']:
top_loadings = loadings_df.loc[pc].abs().sort_values(ascending=False).head(5)
print(f"Top |loadings| for {pc}:")
for feat in top_loadings.index:
    print(f" {feat}: {loadings_df.loc[pc, feat]:.3f}")
print()

print('Default rate by PC1 quartile:')
pca_df['PC1_quartile'] = pd.qcut(pca_df['PC1'], 4, labels=['Q1 (low)', 'Q2', 'Q3', 'Q4 (high)'])
display(pca_df.groupby('PC1_quartile', observed=True)['default'].mean().mul(100))
```

Top |loadings| for PC1:

fico: 0.502
int.rate: -0.491
revol.util: -0.417
credit.policy: 0.307
dti: -0.274

Top |loadings| for PC2:

log.annual.inc: 0.542
installment: 0.470
days.with.cr.line: 0.420
revol.bal: 0.411
fico: 0.201

Top |loadings| for PC3:

inq.last.6mths: 0.496
purpose_debt_consolidation: -0.491
credit.policy: -0.445
purpose_home_improvement: 0.249
revol.util: -0.246

Default rate by PC1 quartile:

PC1_quartile

Q1 (low)	24.34
Q2	17.79
Q3	13.95
Q4 (high)	7.93

Name: Default rate (%), dtype: float64

8. Summary of findings

- **Class balance:** 16.0% of loans defaulted (1,533 of 9,578), matching the data description; `debt_consolidation` is the most common loan purpose and 80.5% of loans meet LendingClub's credit policy (`credit.policy` = 1).
- **Numerical distributions:** `revol.bal` , `dti` , `days.with.cr.line` , and `inq.last.6mths` are right-skewed with long tails / outliers worth screening; `fico` and `log.annual.inc` are closer to symmetric/bell-shaped; `int.rate` is roughly bell-shaped between 6% and 22%.
- **Correlations:** `int.rate` is negatively correlated with `fico` and positively correlated with `revol.util` , confirming LendingClub prices risk into the rate (per the case notes). The KPI fields (`tot.payment` , `principal` , `interest`) are, by construction, almost perfectly correlated with `installment / int.rate` — confirming these should be excluded from predictive/PCA feature sets, as done here.
- **PCA:** with `purpose` one-hot encoded, the 18 standardized predictor features spread their variance more evenly than a small numeric-only dataset — PC1 explains 15.3%, PC2 11.5%, PC3 8.9%; it takes about 13 components to reach 90% cumulative explained variance. This means no single component dominates, i.e. loan risk is genuinely multi-dimensional.

- **Interpreting the top components:**
 - **PC1** ("risk/pricing axis"): high positive loading on `fico` , negative on `int.rate` , `revol.util` , and `dti` , positive on `credit.policy` . High PC1 = safer borrower (high FICO, low utilization, meets credit policy, low rate).
 - **PC2** ("income/credit-history size axis"): positive loading on `log.annual.inc` , `installment` , `days.with.cr.line` , and `revol.bal` — separates borrowers by income level and credit-history length/size rather than riskiness per se.
 - **PC3**: dominated by `inq.last.6mths` (positive) vs. `purpose_debt_consolidation` and `credit.policy` (negative) — a recent-credit-shopping / loan-purpose axis.
- **Biplot / default overlay**: defaulted loans (red) concentrate toward low PC1 values, and the default rate falls monotonically across PC1 quartiles: **24.3% (lowest PC1 quartile) → 17.8% → 14.0% → 7.9% (highest PC1 quartile)**. This confirms PC1 behaves as a genuine risk score built purely from unsupervised structure in the predictors, even though there is meaningful overlap between classes (PC1 alone does not perfectly separate defaults from non-defaults).